Microsoft Fabric End-to-End Earthquake Analytics Pipeline

USGS Data | Medallion Architecture | DP-700 Portfolio Project

---

1. Executive Summary

This project demonstrates an end-to-end data engineering solution built using Microsoft Fabric. The objective was to design and implement a scalable medallion architecture pipeline that ingests real-time earthquake data from the USGS API, processes it through structured layers, and produces analytics-ready datasets for reporting and insights.

It incorporates real-world data engineering practices including incremental ingestion, schema enforcement, and layered data modeling using the medallion architecture.

The solution leverages Lakehouse storage, PySpark notebooks, Delta tables, and pipeline orchestration to simulate a production-grade data platform.

2. Business and Technical Objective

Business Objective:

- Enable analysis of earthquake activity trends across time and regions
- Provide structured datasets for reporting and decision-making
- Simulate real-world ingestion of external event-driven data

Technical Objective:

- Build a medallion architecture (Bronze, Silver, Gold)
- Implement incremental data ingestion from an API
- Enforce schema consistency and deduplication
- Use Delta tables for reliable data storage
- Orchestrate workflows using Fabric pipelines

3. Architecture Overview

Data Flow

USGS API → Bronze Layer (Raw JSON Files) → Silver Layer (Flattened Delta Table) → Gold Layer (Aggregated Tables) → Reporting Layer

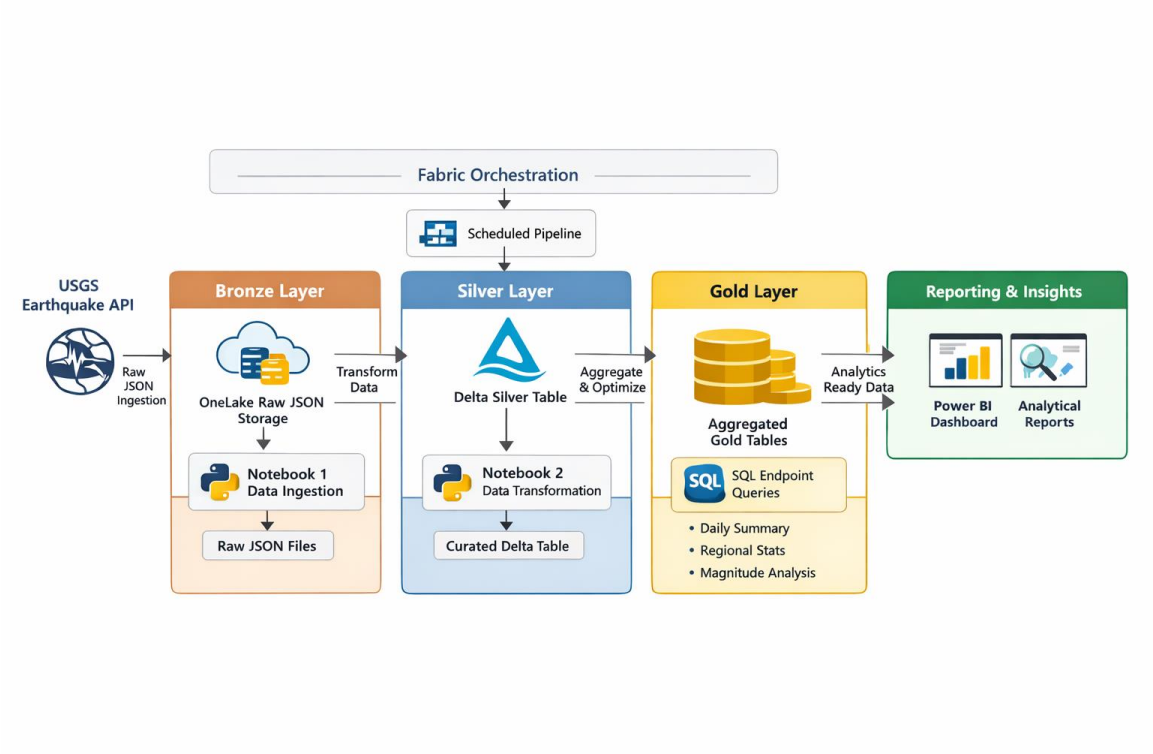
Components Used

- Microsoft Fabric Lakehouse
- PySpark Notebooks
- Delta Tables
- Data Pipeline

- SQL Endpoint (for querying curated data)

Orchestration

- 01_bootstrap_bronze.ipynb (notebook): Bronze ingestion
- 02_incremental_ingestion.ipynb (notebook): Bronze ingestion
- 03_bronze_to_silver.ipynb (notebook): Bronze to Silver transformation
- 04_silver_to_gold.ipynb (notebook): Silver to Gold transformation
- Pl_usgs (Data Pipeline): Scheduled hourly execution



4. Data Engineering Implementation

Bronze Layer (Raw Ingestion)

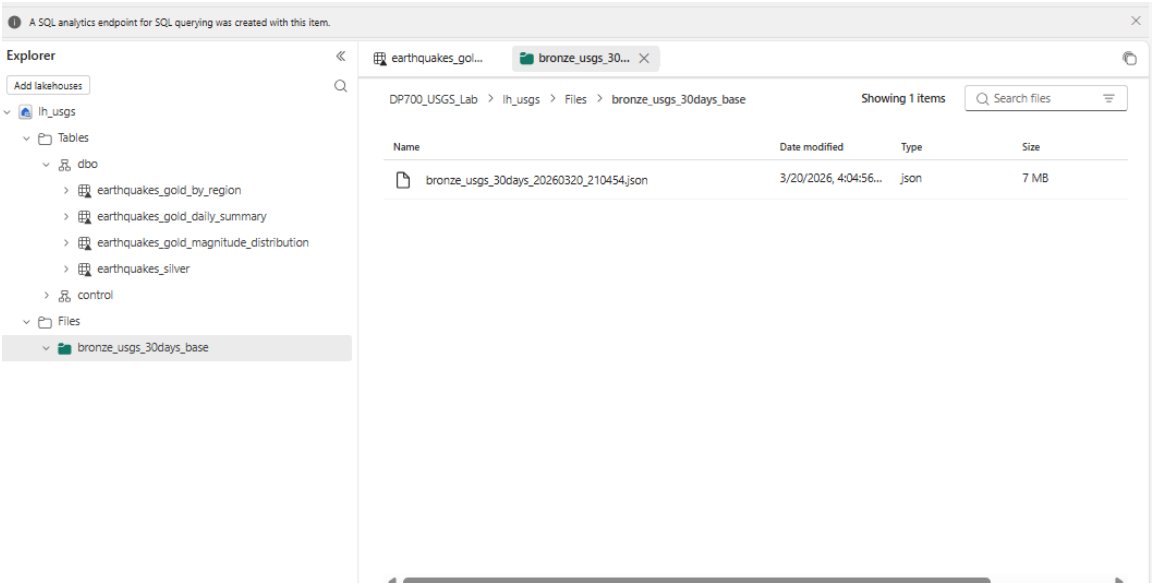
- Data source: USGS Earthquake API
- Format: JSON (GeoJSON structure)
- Storage: OneLake Files/bronze
- Approach:
  - Initial bootstrap for historical data (30 days)
  - Hourly ingestion using API feed

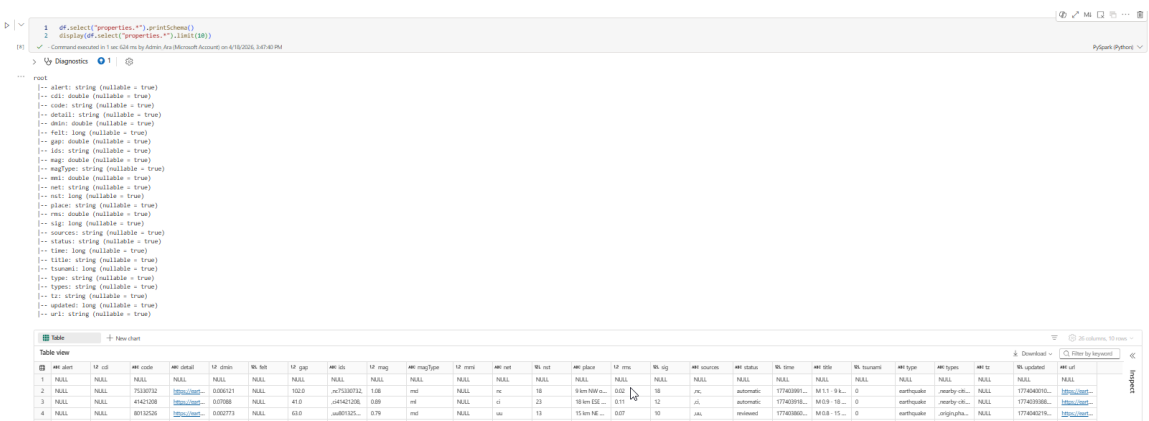
Output:

Raw JSON files stored with timestamps

Bronze Layer Storage Design

- Raw USGS earthquake data is ingested and stored as JSON files in the Bronze layer.
- A bootstrap load captures historical data (30-day snapshot).
- Incremental hourly ingestion was implemented but designed to process and remove files after ingestion to avoid duplication and reduce storage footprint.
- File naming convention includes timestamp to track ingestion time.





```python
from pyspark.sql.functions import explode

df = spark.read.json("Files/bronze_usgs_30days_base/*.json")
df.printSchema()
```

[6] ✓ - Command executed in 1 sec 376 ms by Admin Ara (Microsoft Account) on 4/18/2026, 3:43:56 PM

```
root
 |-- _corrupt_record: string (nullable = true)
 |-- geometry: struct (nullable = true)
 |    |-- coordinates: array (nullable = true)
 |    |    |-- element: double (containsNull = true)
 |    |-- type: string (nullable = true)
 |-- id: string (nullable = true)
 |-- properties: struct (nullable = true)
 |    |-- alert: string (nullable = true)
 |    |-- cdi: double (nullable = true)
 |    |-- code: string (nullable = true)
 |    |-- detail: string (nullable = true)
 |    |-- dmin: double (nullable = true)
 |    |-- felt: long (nullable = true)
 |    |-- gap: double (nullable = true)
 |    |-- ids: string (nullable = true)
 |    |-- mag: double (nullable = true)
 |    |-- magType: string (nullable = true)
 |    |-- mmi: double (nullable = true)
 |    |-- net: string (nullable = true)
 |    |-- nst: long (nullable = true)
 |    |-- place: string (nullable = true)
 |    |-- rms: double (nullable = true)
 |    |-- sig: long (nullable = true)
 |    |-- sources: string (nullable = true)
 |    |-- status: string (nullable = true)
 |    |-- time: long (nullable = true)
 |    |-- title: string (nullable = true)
 |    |-- tsunami: long (nullable = true)
 |    |-- type: string (nullable = true)
 |    |-- types: string (nullable = true)
 |    |-- tz: string (nullable = true)
 |    |-- updated: long (nullable = true)
 |    |-- url: string (nullable = true)
 |-- type: string (nullable = true)
```

Silver Layer (Data Transformation)

- **Purpose**:
  - Flatten nested JSON structure
  - Enforce schema consistency
  - Prepare structured dataset
- **Key Transformations**:
  - Flattened nested JSON structure by extracting fields from properties and geometry
  - Extracted fields:
    - event_id
    - magnitude
    - place
    - time
    - coordinates (longitude, latitude, depth)
  - Converted timestamps to readable format
- **Deduplication**:

- MERGE based on event_id

- **Output Table:**
  - earthquakes_silver (Delta table)

```python
df_gold_region = spark.table("earthquakes_gold_by_region")
df_gold_region.printSchema()
df_gold_region.show(10)
display(df_gold_region.limit(10))
```

[21] ✓ - Command executed in 2 sec 979 ms by Admin_Ara (Microsoft Account) on 4/18/2026, 4:09:04 PM

```
root
 |-- place: string (nullable = true)
 |-- total_earthquakes: long (nullable = true)
 |-- avg_magnitude: double (nullable = true)
 |-- max_magnitude: double (nullable = true)

+--------------------+-----------------+------------------+-------------+
|               place|total_earthquakes|     avg_magnitude|max_magnitude|
+--------------------+-----------------+------------------+-------------+
|18 km ESE of Silv...|               24| 2.222916666666667|         3.62|
|9 km NW of The Ge...|               55|0.7814545454545455|         1.96|
|58 km S of Whites...|               52|1.8499999999999999|          3.0|
|24 km WNW of Ment...|               26|1.8038461538461534|          2.8|
|17 km SE of Silve...|               45|1.8762222222222225|          3.3|
|57 km S of Whites...|               61| 1.960655737704918|          3.5|
|3 km NW of The Ge...|               66|0.9859090909090906|         2.86|
|56 km S of Whites...|               67|1.9492537313432838|          3.6|
|23 km WNW of Ment...|               21|1.6952380952380952|          2.5|
|9 km WNW of Cobb, CA|               28|0.9596428571428574|         2.82|
+--------------------+-----------------+------------------+-------------+
only showing top 10 rows
```

Table ··· + New chart

Table view

	place	total_earthquakes	avg_magnitude	max_magnitude
1	18 km ESE ...	24	2.222916666666667	3.62
2	9 km NW o...	55	0.7814545454545455	1.96
3	58 km S of ...	52	1.8499999999999999	3.0
4	24 km WN...	26	1.8038461538461534	2.8

Gold Layer (Analytics Ready Data)

Purpose:

Provide aggregated datasets for reporting

Tables Created:

- Daily summary (event counts, avg magnitude)
- Region-based aggregation
- Magnitude distribution

Example Metrics:

- Total earthquakes per day
- Maximum magnitude
- Regional activity

```python
df_gold_region = spark.table("earthquakes_gold_by_region")
df_gold_region.printSchema()
df_gold_region.show(10)
display(df_gold_region.limit(10))
```

[21] ✓ - Command executed in 2 sec 979 ms by Admin_Ara (Microsoft Account) on 4/18/2026, 4:09:04 PM

```
root
 |-- place: string (nullable = true)
 |-- total_earthquakes: long (nullable = true)
 |-- avg_magnitude: double (nullable = true)
 |-- max_magnitude: double (nullable = true)

+--------------------+-----------------+------------------+-------------+
|               place|total_earthquakes|     avg_magnitude|max_magnitude|
+--------------------+-----------------+------------------+-------------+
|18 km ESE of Silv...|               24| 2.222916666666667|         3.62|
|9 km NW of The Ge...|               55|0.7814545454545455|         1.96|
|58 km S of Whites...|               52|1.8499999999999999|          3.0|
|24 km WNW of Ment...|               26|1.8038461538461534|          2.8|
|17 km SE of Silve...|               45|1.8762222222222225|          3.3|
|57 km S of Whites...|               61| 1.960655737704918|          3.5|
|3 km NW of The Ge...|               66|0.9859090909090906|         2.86|
|56 km S of Whites...|               67|1.9492537313432838|          3.6|
|23 km WNW of Ment...|               21|1.6952380952380952|          2.5|
|9 km WNW of Cobb, CA|               28|0.9596428571428574|         2.82|
+--------------------+-----------------+------------------+-------------+
only showing top 10 rows
```

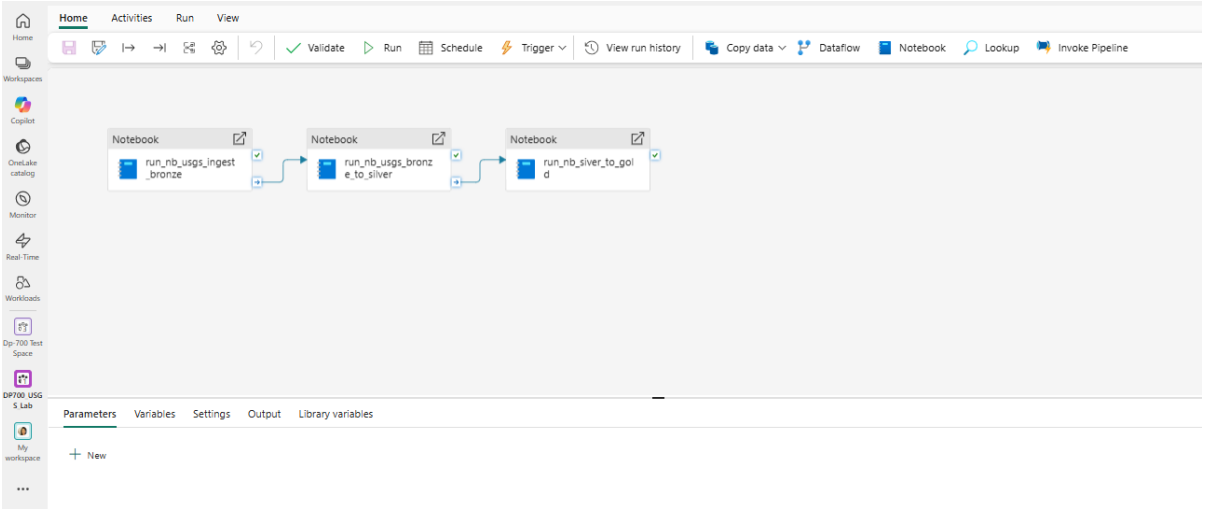
Table | New chart

Table view | Download

	place	total_earthquakes	avg_magnitude	max_magnitude
1	18 km ESE ...	24	2.222916666666667	3.62
2	9 km NW o...	55	0.7814545454545455	1.96
3	58 km S of ...	52	1.8499999999999999	3.0
4	24 km WN...	26	1.8038461538461534	2.8

Pipeline Orchestration

- Built a Fabric pipeline to automate execution
- Steps:
    - ➢ Run ingestion notebook
    - ➢ Run transformation notebook
- Frequency:
    - ➢ Hourly execution



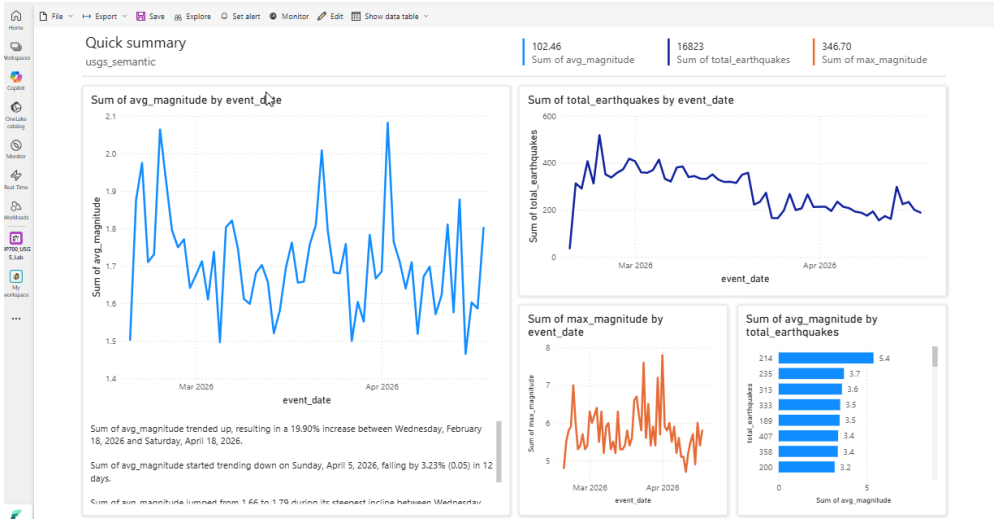
5. Analytics and Reporting Layer

The curated datasets enable analysis such as:

- Earthquake frequency trends over time
- Magnitude distribution analysis
- Regional activity comparison
- Recent seismic activity monitoring

Data Consumption Options:

- Integration with Power BI for visualization
- Direct querying via SQL endpoint

- 

6. Repository Assets (GitHub Structure)

Project Structure

```
fabric-usgs-earthquake-project/
  ├── README.md
  ├── docs/
  ├── notebooks/
  ├── sql/
  ├── data-samples/
  ├── screenshots/
  └── assets/
```

Included Artifacts

- Exported notebooks (.ipynb)
- SQL scripts for Gold layer
- Sample raw JSON files
- Sample curated outputs
- Architecture diagram
- Screenshots of:
    - Lakehouse
    - Pipeline
    - Tables
    - Notebooks

7. Deployment Pipeline and Environment Management

Purpose:

Provide a structured approach for promoting and managing the solution across environments using Microsoft Fabric deployment pipelines.

Approach:

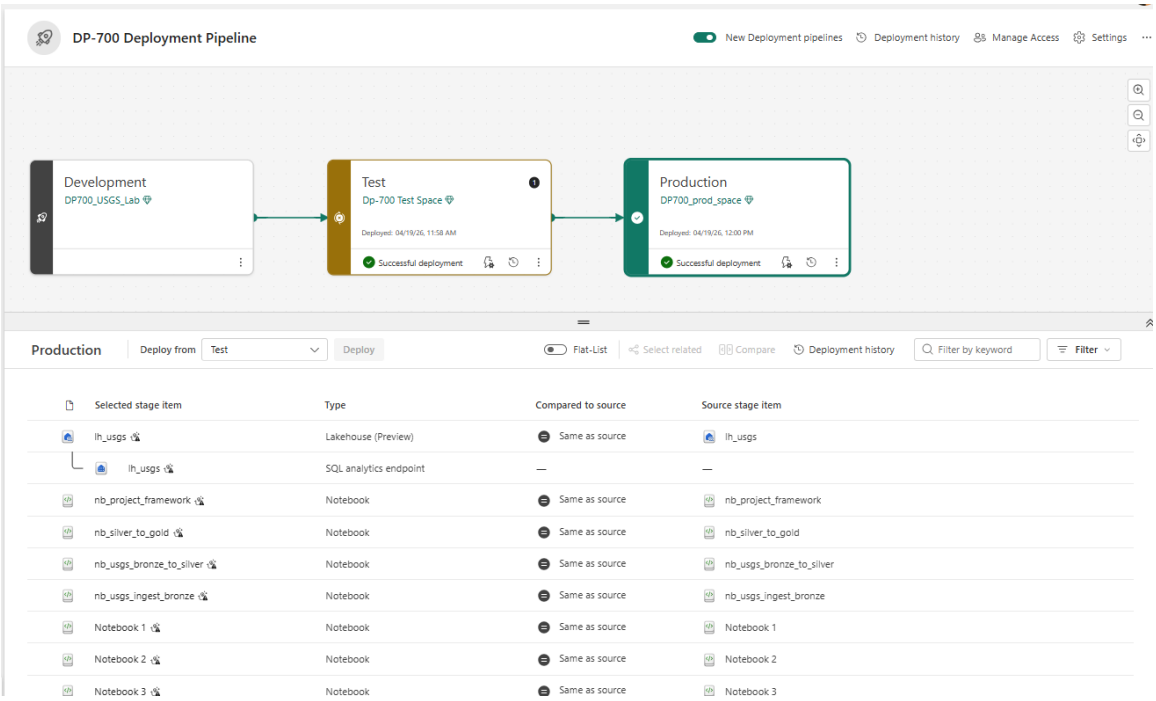
- Utilize Microsoft Fabric deployment pipelines to manage lifecycle stages such as Development, Test, and Production.

- Promote artifacts including Lakehouses, notebooks, data pipelines, and semantic models across environments.
- Maintain consistent solution structure while allowing environment-specific configurations.

Key Considerations:

- Separate development and production workspaces to support controlled deployments.
- Parameterize configurations such as file paths, table names, and connections for environment flexibility.
- Ensure schema consistency across environments to avoid failures during promotion.
- Apply appropriate access control and permissions for each environment.

Deployment Pipeline Overview:



Output:

A deployment-ready architecture aligned with Microsoft Fabric best practices for environment promotion and lifecycle management.

8. Lessons Learned and Improvements

Key Challenges

- Writing files to OneLake using notebook utilities
- Handling schema drift between bootstrap and incremental loads
- Managing incremental ingestion without duplication
- Pipeline parameter passing limitations

Solutions Applied

- Used notebook file utilities for writing JSON
- Enforced schema using existing Silver table structure
- Implemented MERGE logic for deduplication
- Simplified pipeline by reading files instead of passing parameters

Future Improvements

- Add real-time streaming ingestion
- Implement monitoring and alerting
- Enhance data validation layer
- Integrate advanced analytics or machine learning

Final Notes

This project demonstrates practical implementation of:

- Medallion architecture
- Incremental data ingestion
- Data transformation using PySpark
- Delta Lake best practices
- Pipeline orchestration in Microsoft Fabric

It reflects real-world data engineering design and execution aligned with modern analytics platforms.